

CNN, RNN, GAN, and Transformer

A plain-language companion to Session 2, slide 16

These names describe ways neural-network systems can be organized or trained. You do not need to build the mathematics behind them yet. At this stage, learn to recognize:

1. what kind of information each one is designed to handle;
2. what basic idea makes it work;
3. where it is commonly used; and
4. what question you should ask when someone proposes it.

The four ideas in one minute

Name	Plain-language idea	Common fit	Memory phrase
CNN	Scans nearby regions to find small patterns, then combines them into larger patterns	Images and spatial data	Scan local patterns
RNN	Reads a sequence one step at a time while carrying information forward	Ordered sequences and streams	Carry memory forward
GAN	Trains a generator and a judge against each other	Creating synthetic examples	Creator versus critic
Transformer	Uses attention to decide which parts of the input matter to one another	Language, long sequences, and multimodal systems	Connect relevant parts

These are not four levels of intelligence. A Transformer is not automatically better than a CNN, and a neural network is not automatically better than a simpler model. The right choice depends on the task, data, constraints, and evidence.

1. CNN: Convolutional Neural Network

The simple idea

A CNN examines a small region at a time. It looks for simple local patterns such as edges, curves, colors, and textures. Later layers combine those small patterns into larger structures.

```
IMAGE
|
v
[scan small regions]
|
```

```

+--> edges and corners
      |
      v
    shapes and textures
      |
      v
  larger structures
      |
      v
classification or detection

```

A useful analogy is a quality inspector moving a magnifying glass across a product. The inspector first notices a scratch or unusual edge, then considers where it appears and whether the complete object looks defective.

Example

For diabetic-retinopathy screening, a CNN can learn visual patterns associated with damaged blood vessels in retinal photographs. It does not receive a written rule describing every possible damaged vessel. It learns the visual patterns from many labeled images.

Good fits

- medical-image classification;
- production-line defect detection;
- face or object recognition;
- identifying regions inside an image; and
- some audio, video, and spatial-sensor problems.

Limitation to remember

A CNN learns from the examples it receives. If the training images do not represent the cameras, lighting, devices, or populations encountered later, performance may fall.

Question to ask

Are the training images correctly labeled and representative of the images the system will see after deployment?

2. RNN: Recurrent Neural Network

The simple idea

An RNN reads an ordered sequence one step at a time. After each step, it carries a summary of what it has seen into the next step. This carried information is often called its **hidden state** or memory.

```

step 1          step 2          step 3          result
x1 ----> [memory] ----> x2 ----> [memory] ----> x3 ----> output
      ^               ^

```

| |
 what came before updated context

A useful analogy is listening to a sentence word by word. The meaning of the current word depends partly on what you remember from the earlier words.

Example

Suppose a sensor reports temperature every minute. An RNN can process the readings in order and use the recent pattern to predict the next reading or identify an unusual change.

Good fits

- time-ordered sensor readings;
- speech and other streaming signals;
- older language-processing systems;
- compact sequence models; and
- forecasting problems where order matters.

Limitation to remember

Basic RNNs can struggle to retain information from far back in a long sequence. LSTM and GRU networks were created to improve that memory. Transformers have replaced RNNs in much modern language work, but RNNs still appear in existing systems and selected streaming or time-series applications.

Question to ask

How much earlier context must the system remember, and has this model been tested over sequences that long?

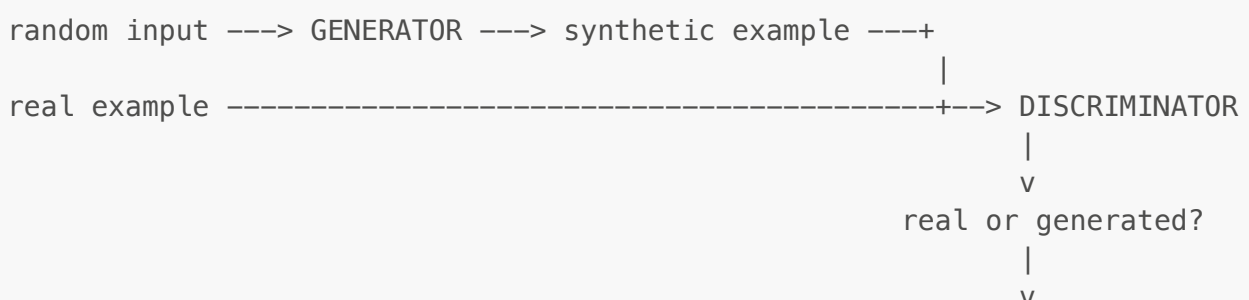
3. GAN: Generative Adversarial Network

The simple idea

A GAN trains two networks together:

- the **generator** creates a synthetic example; and
- the **discriminator** judges whether an example is real or generated.

They improve through competition. The generator tries to fool the discriminator, while the discriminator learns to spot increasingly convincing generated examples.



networks

feedback improves both

A useful analogy is a counterfeiter and an investigator training against each other. Better counterfeits force the investigator to improve; a better investigator forces the counterfeiter to improve.

Example

A manufacturer with too few photographs of rare defects might test a GAN for creating additional synthetic defect images. Those images could supplement training, but experts must verify that they are realistic and do not introduce false patterns.

Good fits

- synthetic-image generation;
- data augmentation;
- image-to-image transformation; and
- simulation or privacy research where realistic synthetic examples are useful.

Limitation to remember

GANs can be difficult to train. A generator may produce only a narrow range of examples, a problem called **mode collapse**. Generated data can also preserve biases or create unrealistic details. Many modern image generators now use diffusion models instead of GANs.

Question to ask

How was the quality and diversity of the generated data validated by people who understand the real domain?

Technical clarification

CNN, RNN, and Transformer usually name architecture families. A GAN more precisely names a **training arrangement** involving a generator and a discriminator. Those two networks can themselves use CNNs, Transformers, or other architectures.

4. Transformer

The simple idea

A Transformer uses **attention** to estimate which parts of the input are relevant to one another. Instead of carrying one running memory through the sequence, it can compare many parts of the input directly.

```
INPUT:  The customer sat by the bank of the river.
          |               ^           ^
          +----- attention links -----+
```

"bank" is interpreted using nearby words such as "river," not only from the word "bank" by itself.

```

tokens or patches
  |
  v
[attention: which parts matter to one another?]
  |
  v
context-aware representations
  |
  v
classify, extract, answer, translate, or generate

```

A useful analogy is reading a paragraph while drawing lines between words and sentences that explain one another. Attention gives stronger weight to the connections that appear useful for the task.

Example

In customer service, a Transformer can read a customer's message, connect it with relevant policy text, and draft a response. A human still needs to verify important facts and decisions because fluent language is not the same as truth.

Good fits

- language understanding and generation;
- document classification and extraction;
- translation and summarization;
- long-range sequence relationships; and
- modern systems that combine text, images, audio, or video.

Limitation to remember

Transformers can require substantial data and computing resources. A generated answer may sound convincing while being incorrect. Attention is a learned calculation, not human understanding or proof of causation.

Question to ask

What trusted evidence grounds the output, and how will important claims be checked?

A side-by-side visual

CNN
local scan

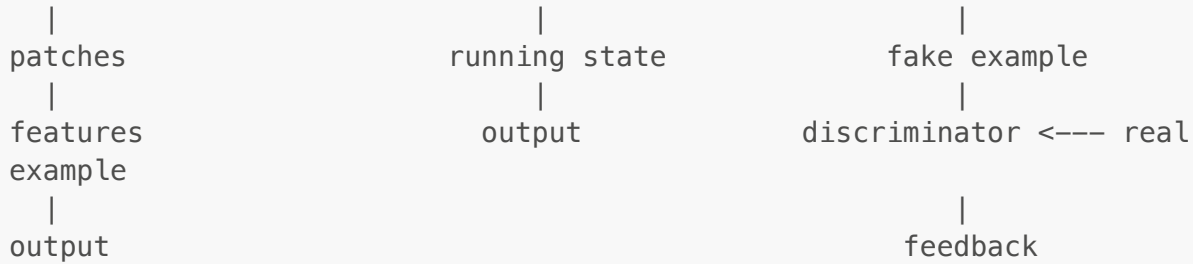
pixels

RNN
ordered memory

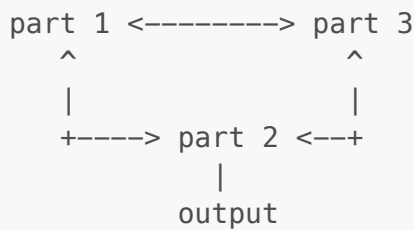
x1 -> x2 -> x3

GAN
competitive training

noise -> generator



TRANSFORMER
attention across relevant parts



The same task can use different model families

Do not choose a model from the business task alone. **Data modality** matters.

Business task	Input data	Reasonable starting point
Classify a retinal image	Image	CNN or Vision Transformer
Classify a customer email	Text	Transformer
Forecast from an ordered sensor stream	Time series	RNN, Transformer, or a dedicated forecasting model
Create synthetic defect photographs	Images to generate	GAN or diffusion model
Score card transactions for fraud	Structured table	Tree ensemble such as random forest or gradient boosting

The current Applied AI Studio fraud lab uses a **random forest**, not one of these four neural-network families. That is intentional. Its inputs are structured, named columns, and a tree ensemble is fast, effective, and easier to explain. A more complex model has to earn its complexity through better evidence.

A practical selection guide

What kind of information is central to the problem?

Image with important local patterns?

-> Start by considering a CNN.

Ordered stream where recent state must carry forward?

-> Consider an RNN or a dedicated time-series model.

Language or long-range relationships?

-> Consider a Transformer.

Need to generate synthetic examples through adversarial training?

-> A GAN may be relevant; compare it with diffusion methods.

Rows and columns with named business features?

-> Start with linear models or tree ensembles, not automatically deep learning.

What students should remember

1. **CNN**: Finds local patterns and builds them into larger visual structures.
2. **RNN**: Processes a sequence in order and carries a changing memory forward.
3. **GAN**: Trains a creator and a critic together to generate synthetic examples.
4. **Transformer**: Uses attention to connect relevant parts of an input.
5. The data format helps determine the model family.
6. More complex does not automatically mean more accurate, safer, or more appropriate.

Five-minute class activity (Optional)

For each scenario, choose the strongest first candidate and explain your choice in one sentence.

1. Find surface scratches from production-line camera images.
2. Interpret a long customer complaint and draft a reply grounded in company policy.
3. Generate synthetic examples of a rare product defect.
4. Analyze an ordered stream of machine-vibration readings.
5. Score a payment using amount, merchant category, time, and account-history columns.

Suggested answers

1. **CNN**, because local visual patterns such as edges and scratches matter.
2. **Transformer**, because language and relationships across a long document matter.
3. **GAN** or a diffusion model, because the goal is synthetic generation.
4. **RNN**, Transformer, or a dedicated time-series model, because order over time matters.
5. **Tree ensemble**, because the input is structured tabular data; a neural network is not the automatic first choice.